Final Project guidelines for Python

Implementation

1. Use your custom data preprocessing methods

a. Complete ***Preprocessor class*** in *preprocessor.py*
Use your custom data preprocessing functions in project #1 as the methods in Preprocessor class.

```
4   class Preprocessor:
5       """ TODO """
6       def __init__(self, df):
7           # Initialize the preprocessor with a DataFrame
8           self.df = df
```

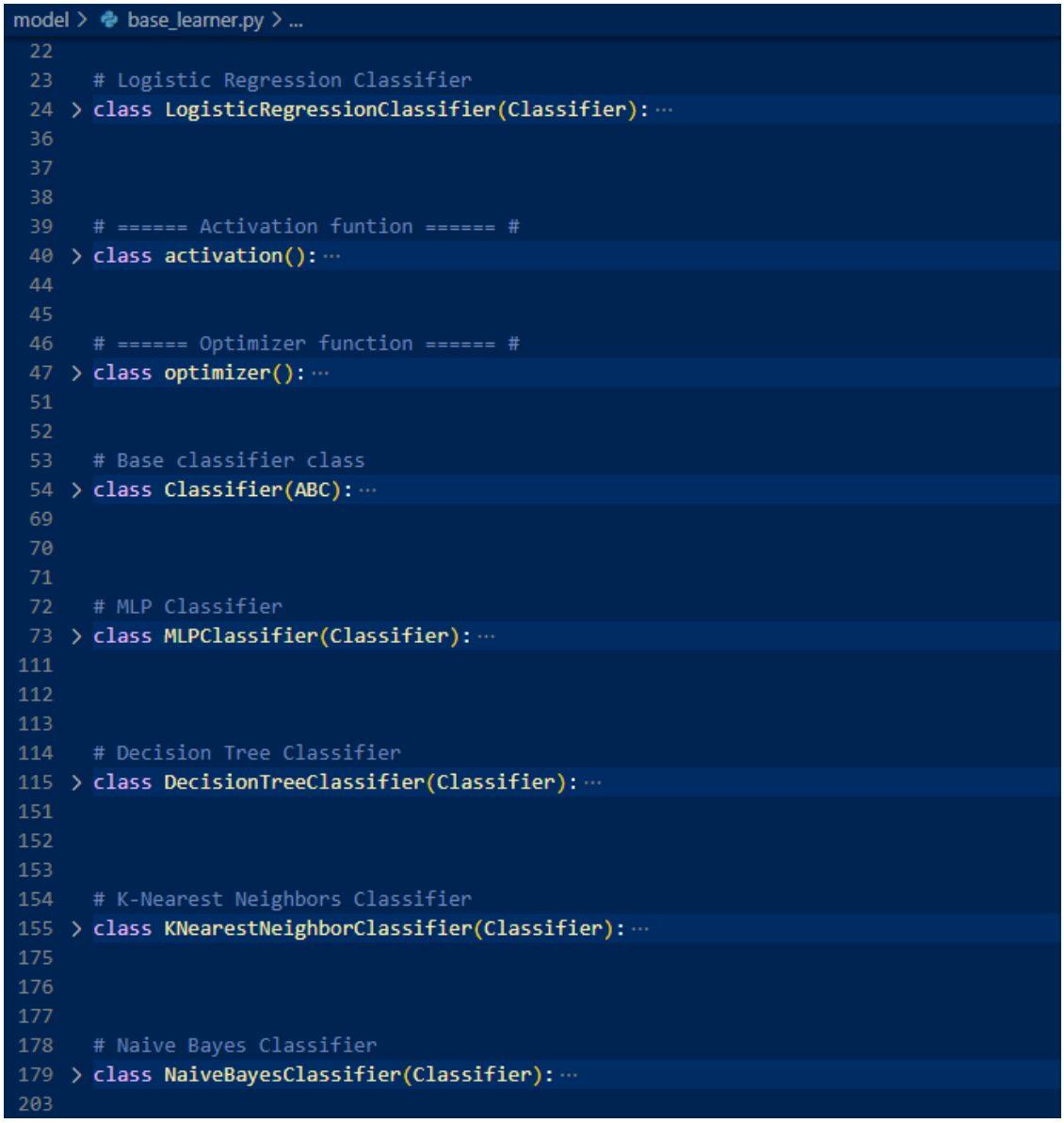
b. Complete ***DataPreprocessing function*** in *main.py*
Use the custom methods in project #1 you implemented to complete dataPreprocessing function.

```
11  def dataPreprocessing():
12      """ TODO, use your own dataPreprocess function here. """
13
14      return
```

Note: You can either use your data preprocessing methods in project #1 or implement a new one for final project.

2. Put the models into “model/base_learner.py”

a. Put the models you implemented in previous project into *“model/base_learner.py”*.



3. Complete Stacking in “model/meta_learner.py”

a. Implement necessary methods in StackingClassifier

model > meta_learner.py > ...

```python
import numpy as np
from sklearn.model_selection import KFold
from base_learner import LogisticRegressionClassifier, MLPClassifier,   \
                        DecisionTreeClassifier, KNearestNeighborClassifier,     \
                        NaiveBayesClassifier
from tqdm import tqdm


class StackingClassifier:
    def __init__(self):
        """
        Stacking Classifier
        """
        pass

    def fit(self, X_train, y_train):
        """
        Fit the stacking model.
        :param X_train: Training data.
        :param y_train: Training labels.
        """
        pass

    def predict(self, X_test):
        """
        Predict using the stacking model.
        :param X_test: Test data.
        :return: Final predictions.
        """
        pass

```

4. Complete main function in *main.py*

a. Build your Stacking, print the result of your K-Fold CV, and save the predict label as .csv file

```
18  def main():
19      root_path = r"yourtraining_data" # change the root path
20      train_X = os.path.join(root_path, "train_x.csv")
21      train_y = os.path.join(root_path, "train_y.csv")
22      test_X = os.path.join(root_path, "test_x.csv")
23
24      fold = KFold(n_splits=10, random_state=42, shuffle=True)
25      # TODO
26      # implement K-Fold CV
27
28      model = StackingClassifier()
29      model.fit(train_X, train_y)
30
31      # TODO
32      # build your Stacking model
33      # predict the output of the testing data
34      # remember to paste the result of K-fold CV to your report
35      # remember to save the predict label as .csv file
36
```

Model Test

We will use “model_test.py” to run the model test for you. Make sure your name of model classes are same as I provided for you in the framework.

```
model_test.py > ...
4   from model.base_learner import LogisticRegressionClassifier, DecisionTreeClassifier, \
5                                  KNearestNeighborClassifier, NaiveBayesClassifier, MLPClassifier
6   from model.meta_learner import StackingClassifier
```

Remember to modify the hyperparameter to pass the test.

e.g.

```
class LogisticRegressionClassifier(Classifier):
    def __init__(self, C=1.0, penalty='l2', lr=1e-2, iterations=600):
```

```
class KNearestNeighborClassifier(Classifier):
    def __init__(self, k=3, distance_metric='euclidean', p=3):
```

The shape of the data is the same as the dataset in previous project.

```
The shape of X is: (100, 77)
The shape of y is: (100,)
```

If everything is functioning correctly, your terminal should display output like this. *(You don’t have to modify anything in model_test.py)*

```
Total # Base learner passed: 5 / 5

Meta learner passed!

All tests completed.
```